

CEN206 Object-Oriented Programming

Week-7 (UMPLE - Part 2)

Spring Semester, 2025-2026

Download [DOC-PDF](#), [DOC-DOCX](#), [SLIDE](#)



Module A: Modeling Exercises & Simple Patterns

Module Outline

- Modeling Exercises
- Simple Patterns in UMPLE (Singleton, Delegation, etc.)

Modeling exercises

Modeling Exercise

- Build a class diagram for the following description.
- If you think there are key requirements missing, then add them.
 - A football (soccer) team has players. Each player plays a position. The team plays some games against other teams during each season. The system needs to record who scored goals, and the score of each game.

Simple patterns (if time)

Singleton pattern

- Standard pattern to enable only a single instance of a class to be created.
 - `private` constructor
 - `getInstance()` method
- Declaring in Uml

```
class University {  
    singleton;  
    name;  
}
```

Delegation pattern

- A class calls a method in its "neighbour"

```
class RegularFlight {  
    flightNumber;  
}  
  
Class SpecificFlight {  
    * -- 1 RegularFlight;  
    flightNumber = {getRegularFlight().getFullNumber()}  
}
```

- Full details of this example in the user manual

Basic constraints

- Shown in square brackets
 - Code is added to the constructor and the set method

```
class X {  
  Integer i;  
  [! (i == 10)]  
}
```

- We will see constraints later in state machines

Takeaway: Modeling Exercises & Simple Patterns

- Practice exercises reinforce UML-to-UMPLE modeling skills
- UMPLE supports common design patterns directly in the language
- Simple patterns like Singleton and Delegation can be expressed concisely in UMPLE
- Hands-on modeling is essential for mastering model-driven development

Module B: Basic State Machines

Module Outline

- Basic State Machines in UMPLE
- States, Transitions, Events
- Guards and Actions
- Nested States

Basic state machines

- <http://statemachines.umple.org>

Basics of state machines

- At any given point in time, the system is in one state.
- It will remain in this state until an event occurs that causes it to change state.
- A state is represented by a rounded rectangle containing the name of the state.
- Special states:
 - A black circle represents the *start state*
 - A circle with a ring around it represents an *end state*

Garage door state machine

```
class GarageDoor{
    status {
        Open {
            buttonOrObstacle -> Closing;
        }
        Closing {
            buttonOrObstacle -> Opening;
            reachBottom -> Closed;
        }
        Closed {
            buttonOrObstacle -> Opening;
        }
        Opening {
            buttonOrObstacle -> HalfOpen;
            reachTop -> Open;
        }
        HalfOpen {
            buttonOrObstacle -> Opening;
        }
    }
}
```

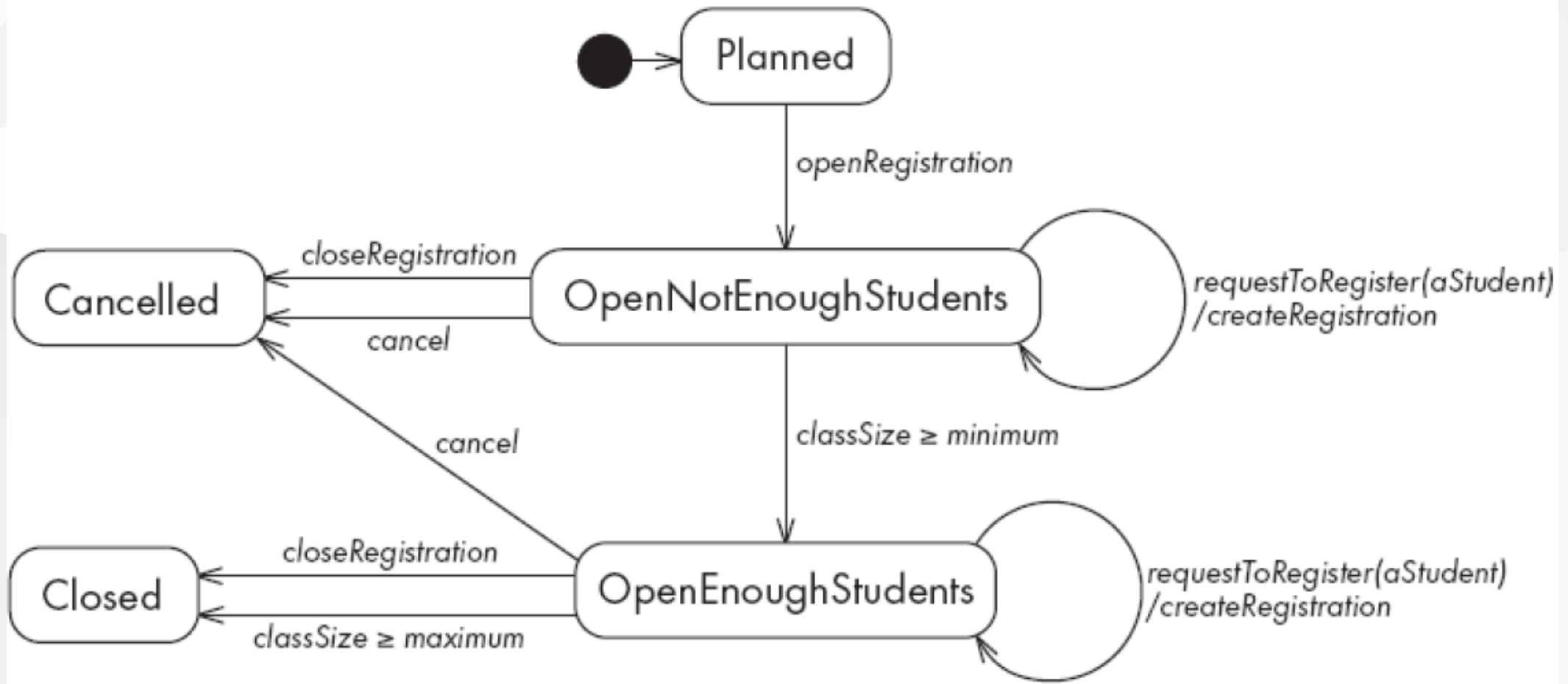
Events

- An occurrence that may trigger a change of state
 - Modeled in Umlle as generated methods that can be called
- Several states may be able to respond to the same event

Transitions

- A change of state in response to an event.
 - It is considered to occur **instantaneously**.
- The label on each transition is the event that causes the change of state.

State diagrams – an example with conditional transitions

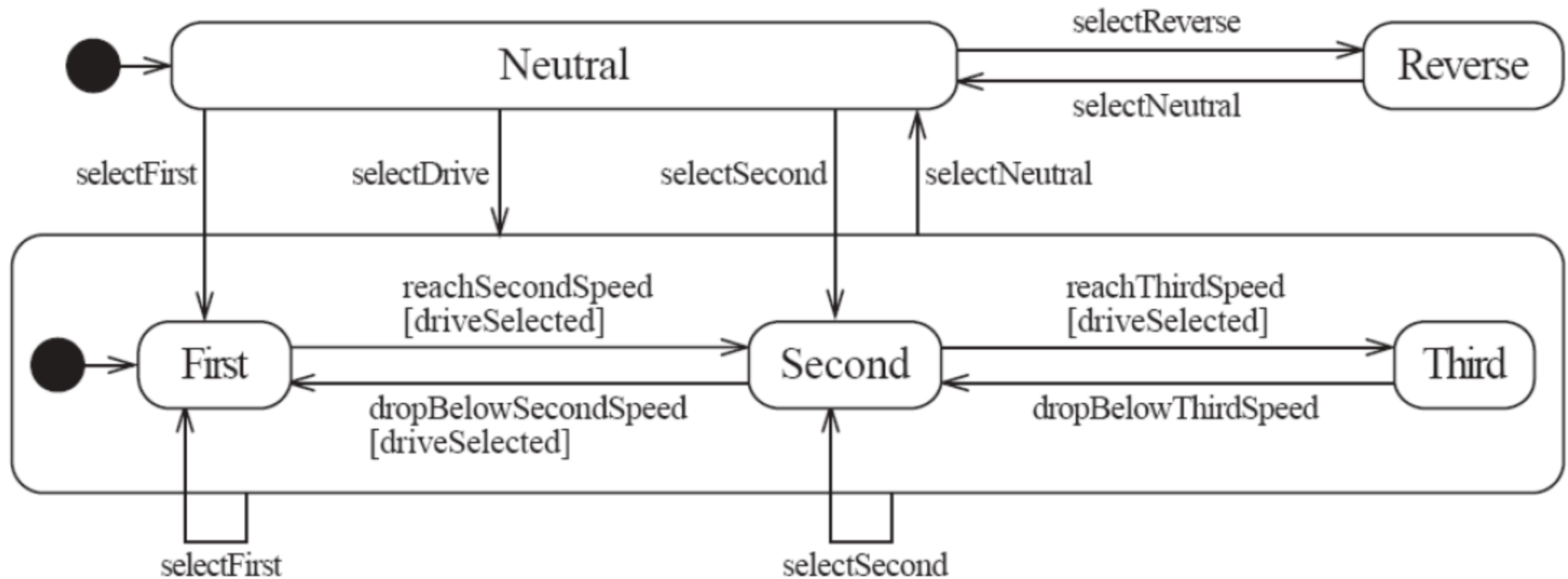


Actions in state diagrams

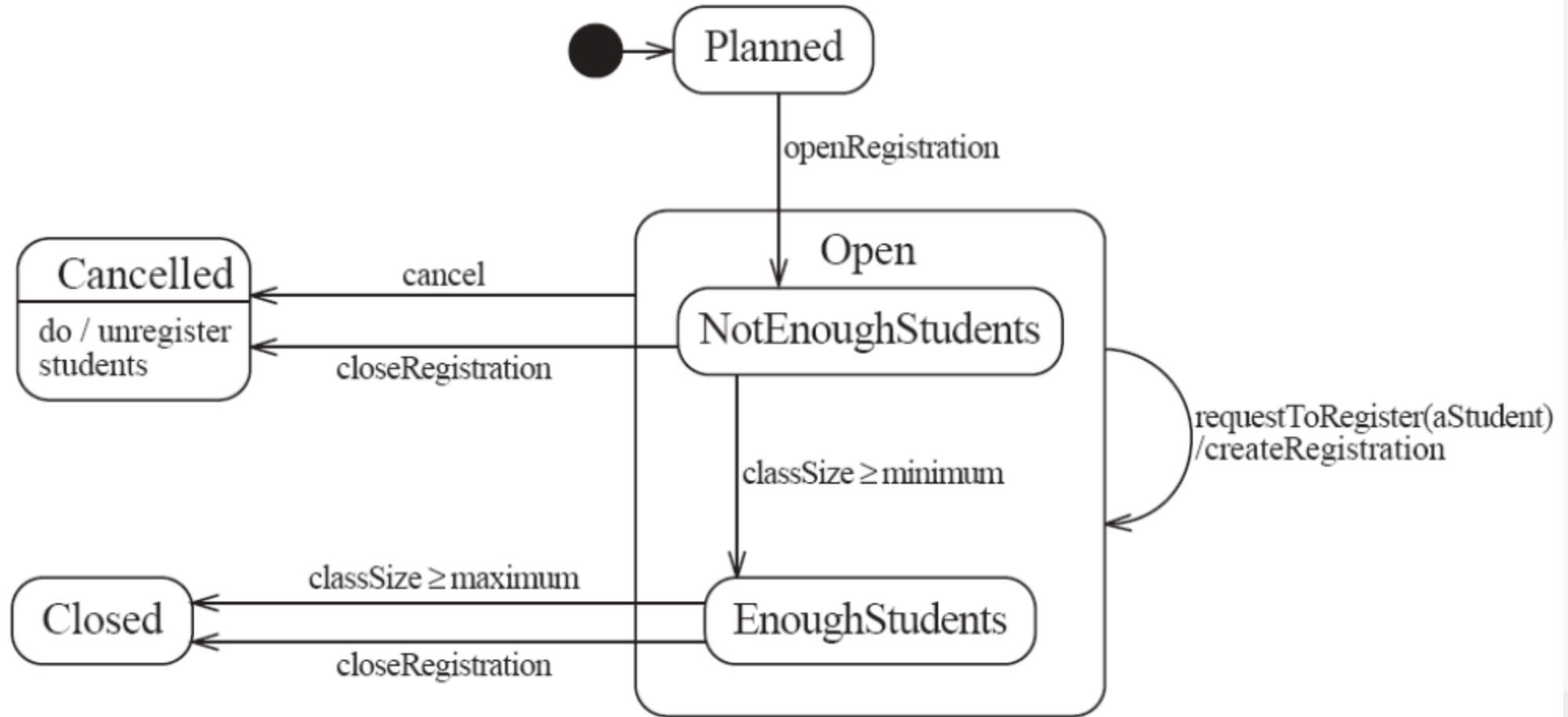
- An action is a block of code that must be executed effectively instantaneously
 - When a particular transition is taken,
 - Upon entry into a particular state, or
 - Upon exit from a particular state
- An action should consume no noticeable amount of time

Nested substates and guard conditions

- A state diagram can be nested inside a state.
 - The states of the inner diagram are called substates.



Nested state diagram – Another example



Auto-transitions

- A transition taken immediately upon entry into a state
 - Unless guarded
- We will look at an example in the user manual

Events with parameters

- Parameters can be referenced in guards and actions.
- We will look at an example in the user manual.

Analysing models

Models can be analysed in several ways

- Visually
- Automatically generated errors and warnings
- State tables (next slide)\
- Metrics
- Formal methods (nuXMV)

State tables and simulations

- Allow analysis of state machines statically without having to write code
- We will explore these in UmpleOnline by looking at state machine examples and generating tables and simulations

Takeaway: Basic State Machines

- UMPLE state machines are defined directly in the class using textual syntax
- States, transitions, and events are first-class constructs in UMPLE
- Guards (conditions) and actions (code blocks) can be attached to transitions
- Nested (hierarchical) state machines enable complex behavior modeling
- UMPLE generates complete state machine implementation code automatically

Module C: Analyzing Models & Concurrency

Module Outline

- Analyzing UMPLE Models
- Concurrency in UMPLE
- State Machine Case Study

Concurrency

Do activities and concurrency

- A do activity executes
 - In a separate thread
 - Until
 - Its method terminates, or
 - The state needs to exit (killing the thread)
- Example uses:
 - Outputting a stream (e.g. playing music)
 - Monitoring something
 - Running a motor while in the state
 - Achieving concurrency, using multiple do activities

Active objects

- These start in a separate thread as they are instantiated.
- Declared with the keyword

active

Default threading in state machines

- As discussed so far, code generated for state machines has the following behaviour:
 - A single thread:
 - Calls an event
 - Executes the event (running any actions)
 - Returns to the caller and continues
- This has two problems:
 - If another thread calls the event at the same time they will **interfere**
 - There can be **deadlocks** if an action itself triggers an event

Queued state machines

- Solve the threading problem:
 - Callers can add events to a queue without blocking
 - A separate thread takes items off the queue 'as fast as it can' and processes them
- Umple syntax: `queued` before the state machine declaration
- *We will look at examples in the manual*

Pooled state machines

- Default Umlle Behavior (including with queued):
 - If an event is received but the system is not in a state that can handle it, then the event is ignored.
- Alternative `pooled` stereotype:
 - Uses a queue (see previous slide)
 - Events that cannot be processed in the current state are left at the head of the queue until a relevant state reached
 - The first relevant event nearest the head of the queue is processed
 - Events may hence be processed out of order, but not ignored

Unspecified pseudo-event

- Matches any event that is not listed
- Can be in any state, e.g.

```
unspecified -> error;
```

Example using unspecified

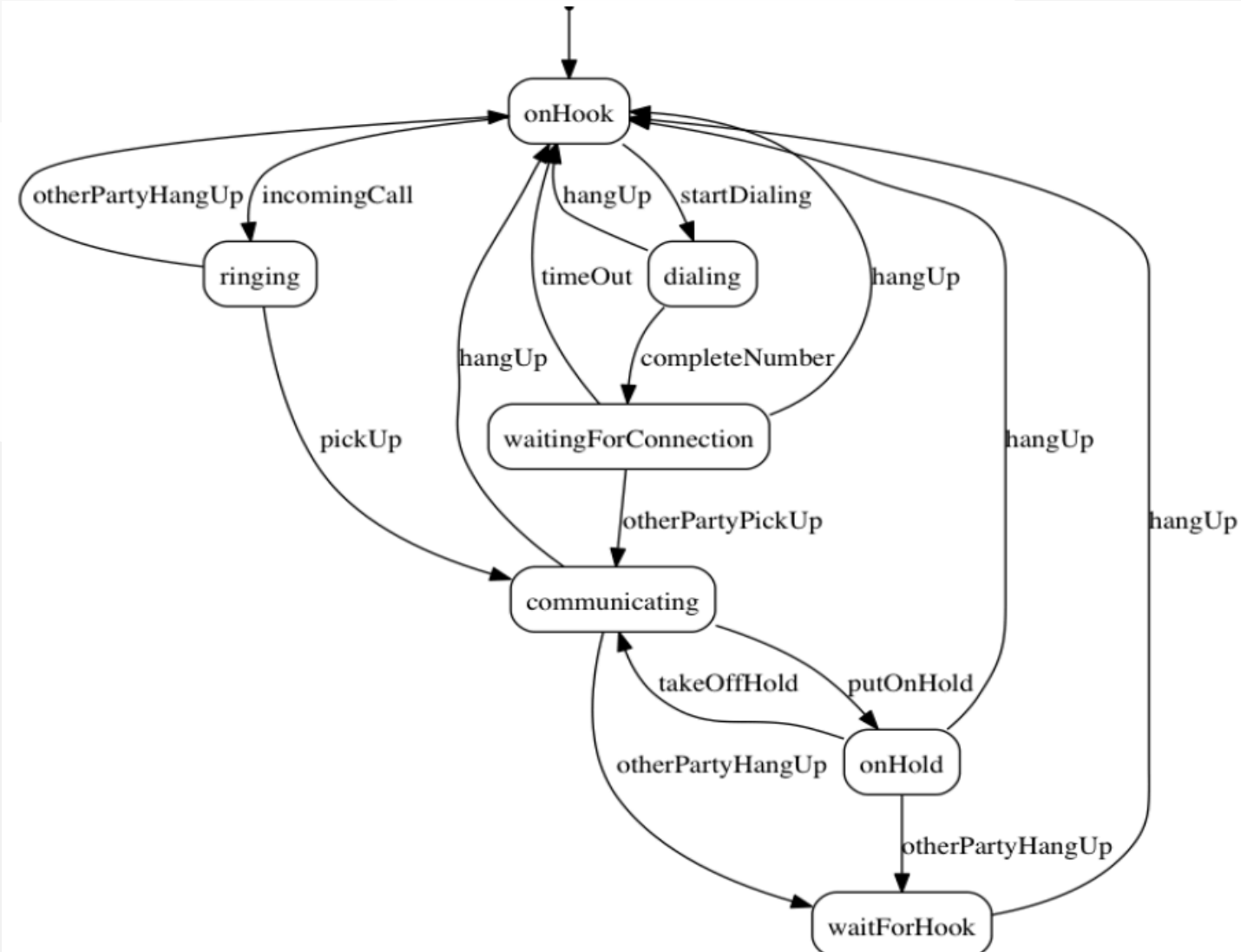
```
class AutomatedTellerMachine{
  queued sm {
    idle {
      cardInserted -> active; maintain -> maintenance;
      unspecified -> error1;
    }
    maintenance { isMaintained -> idle; }
    active {
      entry /{addLog("Card is read");}
      exit /{addLog("Card is ejected");}
      validating {
        validated -> selecting;
        unspecified -> error2;
      }
      selecting {select -> processing; }
      processing {
        selectAnotherTransiction -> selecting;
        finish -> printing;
      }
      printing {receiptPrinted -> idle;}
      cancel -> idle;
    }
    error1 {entry / {printError1();} ->idle;}
    error2 {entry / {printError2();} ->validating;}
  }
}
```

State machines in the user manual

- <http://statemachines.umple.org>

State machine case study

State machine for a phone line



Umple for the phone line example

```
class phone {  
  state {  
    onHook {  
      startDialling -> dialling;  
      incomingCall -> ringing;  
    }  
  
    ringing {  
      pickUp -> communicating;  
      otherPartyHangUp -> onHook;  
    }  
  
    communicating {  
      hangUp -> onHook;  
      otherPartyHangUp -> waitForHook;  
      putOnHold -> onHold;  
    }  
  
    onHold {  
      hangUp -> onHook;  
      otherPartyHangUp -> waitForHook;  
      takeOffHold -> communicating;  
    }  
  }  
}
```

Umple for the phone line example

- con't.

```
dialing {
  completeNumber ->
  waitingForConnection;
  hangUp -> onHook;
}

waitingForConnection {
  otherPartyPickUp -> communicating;
  hangUp -> onHook;
  timeOut -> onHook;
}

waitForHook {
  hangUp -> onHook;
}
}
```

In-class modeling exercise for state machines

- Microwave oven system state machine
 - Events include
 - pressing of buttons
 - door opening
 - door closing
 - timer ending
 - etc.

Takeaway: Analyzing Models & Concurrency

- UMPLE models can be analyzed for consistency and completeness
- Concurrency support: UMPLE can generate thread-safe code from concurrent state machines
- State machine case studies demonstrate real-world application of UMPLE modeling
- Model analysis helps identify design issues before code generation

Module D: Mixins, Aspect Orientation & Traits

Module Outline

- Mixins in UMPLE
- Aspect Orientation
- Traits in UMPLE
- Combining Mixins and Traits

Mixins

Mixins : Motivation

- Product variants have long been important for
 - Product lines/families, whose members target different:
 - hardware, OS, feature sets, basic/pro versions
 - Feature-oriented development (separation of concerns)

Separation of concerns by mixins in Umple

- Mixins allow including attributes, associations, state machines, groups of states, stereotypes, etc
- Example:

```
class X { a; }  
class X { b; }
```

- The result would be a class with both a and b.
- It doesn't matter whether the mixins are
 - Both in the same file
 - One in one file, that includes the other in an other file
 - In two separate files, with a third file invoking them

Typical ways of using mixins

- Separate groups of classes for
 - model (classes, attributes, associations)
 - Methods operating on the model
- Allows a clearer view of the core model
- Another possibility
 - One feature per file

Typical ways of using mixins

- Separate model files (classes, attributes associations)
- ... from files for the same class containing methods
 - Allows a clearer view of the core model
- Separate system features, each into a separate file

Advantages and disadvantages of mixins

- Advantages:
 - Smaller files that are easier to understand
 - Different versions of a class for different software versions (e.g. a professional version) can be built by using different mixins
- Disadvantage
 - Delocalization:
 - Bits of functionality of a class in different files
 - The developer may not know that a mixin exists unless a tool helps show this

Aspect orientation

Aspects : Motivation

- We often don't quite like the code as generated

Or

- We want to do a little more than what the generated code does

Or

- We want to inject some feature (e.g. security checks) into many places of generated or custom code

Aspect orientation : General Concept

- Create a pointcut that specifies (advises) where to inject code at multiple points elsewhere in a system
 - The pointcut uses a pattern
 - Pieces of code that would otherwise be scattered are thus gathered into the aspect
- But: There is potentially acute sensitivity to change
 - If the code changes the aspect may need to change
 - Yet without tool support, developers wouldn't know this
- Drawback : **Delocalization even stronger than for mixins**

Aspect orientation in Umple

- It is common to limit a pointcuts a single class
 - Inject code before, after, or around execution of custom or generated methods and constructors

```
class Person {  
    name;  
    before setName {  
        if (aName != null && aName.length() > 20) { return false;  
        }  
    }  
}
```

- We have found these limited abilities nonetheless solve key problems

Traits

Traits : Motivation

- We may want to inject similar elements into unrelated classes
 - without complex multiple inheritance
- Elements can be
 - Methods
 - Attributes
 - Associations
 - States or state machines
 - .. Anything

Separation of Concerns by Traits

- Allow modeling elements to be made available in multiple classes

```
trait Identifiable {  
  firstName;  
  lastName;  
  address;  
  phoneNumber;  
  fullName = {firstName + " " + lastName}  
  Boolean isLongName() {return lastName.length() > 1;}  
}  
  
class Person {  
  isA Identifiable;  
}
```

- *See more complete version of this in the user manual*

Another Trait example

```
trait T1{
    abstract void method1(); /* required method */
    abstract void method2();
    void method4(){/*implementation - provided method*/ }
}

trait T2{
    isA T1;
    void method3();
    void method1(){/*implementation*/ }
    void method2(){/*implementation*/ }
}

class C1{
    void method3(){/*implementation*/ }
}

class C2{ isA C1; isA T2;
    void method2(){/*implementation*/ }
}
```

Traits With Parameters

```
trait T1< TP isA I1 > {  
  abstract TP method2(TP data);  
  String method3(TP data){ /*implementation*/ }  
}  
interface I1{  
  void method1();  
}  
class C1{ isA I1;  
  isA T1<TP = C1>;  
  void method1(){/*implementation*/}  
  C1 method2(C1 data){ /*implementation*/ }  
}  
class C2{  
  isA I1;  
  isA T1< TP = C2 >;  
  void method1(){/*implementation*/}  
  C2 method2(C2 data){ /*implementation*/ }  
}
```

Trait Parameters in Methods

```
trait T1 <TP>{  
  String method1();  
  String method2(){  
    #TP# instance = new #TP#();  
    return method1() + ":" + instance.process();  
  }  
}  
  
class C1{  
  String process(){/*implementation*/}  
}  
  
class C2{  
  isA T1< TP = C1 >;  
  String method1(){/*implementation*/ }  
}
```

Selecting Subsets of Items in Traits

```
trait T1{
  abstract method1();
  void method2(){/*implementation*/}
  void method3(){/*implementation*/}
  void method4(){/*implementation*/}
  void method5(){/*implementation*/}
}
class C1{
  isA T1<-method2() , -method3(>;
  void method1() {/*implementation related to C1*/}
}
class C2{
  isA T1<+method5(>;
  void method1() {
    /*implementation related to C2*/}
}
```

Renaming Elements when Using Traits

```
trait T1{
  abstract method1();
  void method2(){/*implementation*/}
  void method3(){/*implementation*/}
  void method4(){/*implementation*/}
  void method5(Integer data){/* implementation*/}
}
class C1{
  isA T1< method2() as function2 >;
  void method1() {/*implementation related to C1*/}
}
class C2{
  isA T1< method3() as private function3 >;
  void method1() {/*implementation related to C2*/}
}
class C3{
  isA T1< +method5(Integer) as function5 >;
  void method1() {/*implementation related to C3*/}
}
```


Associations in Traits: Observer Pattern

```
class Dashboard{  
void update (Sensor sensor){ /*implementation*/ }  
}  
class Sensor{  
isA Subject< Observer = Dashboard >;  
}  
trait Subject <Observer>{  
0..1 -> * Observer;  
void notifyObservers() { /*implementation*/ }  
}
```

Using Traits to Reuse State Machines

```
trait T1 {  
  sm1{  
    s0 {e1-> s1;}  
    s1 {e0-> s0;}  
  }  
}  
  
trait T2 {  
  isA T1;  
  sm2{  
    s0 {e1-> s1;}  
    s1 {e0-> s0;}  
  }  
}  
  
class C1 {  
  isA T2;  
}
```

Satisfaction of Required Methods Through State Machines

```
trait T1{
  Boolean m1(String input);
  Boolean m2();
  sm1{
    s1{
      e1(String data) -> /{ m1(data); } s2; }
    s2{
      e2 -> /{ m2(); } s1; }
    }
  }
}

class C1{
  isA T1;
  sm2{
    s1{ m1(String str) -> s2;}
    s2{ m2 -> s1;}
  }
}
```

Changing Name of a State Machine Region

```
trait T1{  
  sm {  
    s1{  
      r1{ e1-> r11; }  
      r11{}  
      ||  
      r2{ e2-> r21; }  
      r21{}  
    }  
  }  
}  
  
class C1{  
  isA T1<sm.s1.r1 as region1,sm.s1.r2 as region2>;  
}
```

Changing the Name of an Event

```
trait T1 {  
  sm1{  
    s0 { e1(Integer index)-> s1;}  
    s1 {e0-> s0;}  
  }  
  sm2{  
    t0 {e1(Integer index)-> t1;}  
    t1 {e0-> t0;}  
  }  
}  
  
class C1 {  
  isA T1<sm1.e1(Integer) as event1, *.e0() as event0>;  
}
```

Mixins and Traits together

- Examples of mixins and traits combined in the user manual:
- Mixins with traits:
 - <https://cruise.umple.org/umple/TraitsandUimpleMixins.html>

Takeaway: Mixins, Aspect Orientation & Traits

- Mixins allow splitting a class definition across multiple files/locations
- Aspect orientation: inject behavior (before/after) into methods without modifying original code
- Traits define reusable sets of attributes, associations, and state machines
- Traits can be parameterized and composed, enabling flexible code reuse
- Mixins + Traits together provide powerful composition mechanisms beyond traditional inheritance

Module E: Mixsets, Testing & Architecture

Module Outline

- Mixsets (Feature-based Composition)
- Unit Testing with UMPLE
- UMPLE Issues List
- Using UMPLE with Builds and CI
- UMPLE's Architecture
- Umplification (Converting Code to UMPLE)
- Conclusion

Mixsets

Mixsets: Motivations

- A feature or variant needs to inject or alter code in many places
 - Historically tools like the C Preprocessor were used
 - Now tools like "Pure: Variants"
- There is also a need to
 - Enable **model variants** in a very straightforward way
 - Blend variants with code/models in core compilers
 - With harmonious syntax + analysable semantics
 - Without the need for tools external to the compiler

Mixsets: Top-Level Syntax

- Mixsets are named sets of mixins

```
mixset Name {  
  // Anything valid in Umler at top level  
}
```

- The following syntactic sugar works for top level elements (class, trait, interface, association, etc.)

```
mixset Name class Classname {  
}
```

Use Statements

- A use statement specifies inclusion of either
 - A file, or
 - A mixset

```
use Name;
```

- A mixset is conceptually a `virtual file` that is composed of a set of model/code elements
- The use statement for a mixset can appear
 - Before, after or among the definition of the mixset parts
 - In `another mixset`
 - On the command line to generate a variant

Mixsets and Mixins: Synergies

- The blocks defined by a mixset are mixins
 - Mixsets themselves can be composed using mixins
 - e.g.

```
mixset Name1 {class X { a; } }
```

- And somewhere else

```
mixset Name1 {class X { b; } }  
use Name1;
```

- Would be the same as:

```
class X { a; b;}
```

Mixset Definitions Internal to a Top-Level Element

```
class X {  
  mixset Name2 {a;}  
  b;  
  
}
```

- Is the same as,

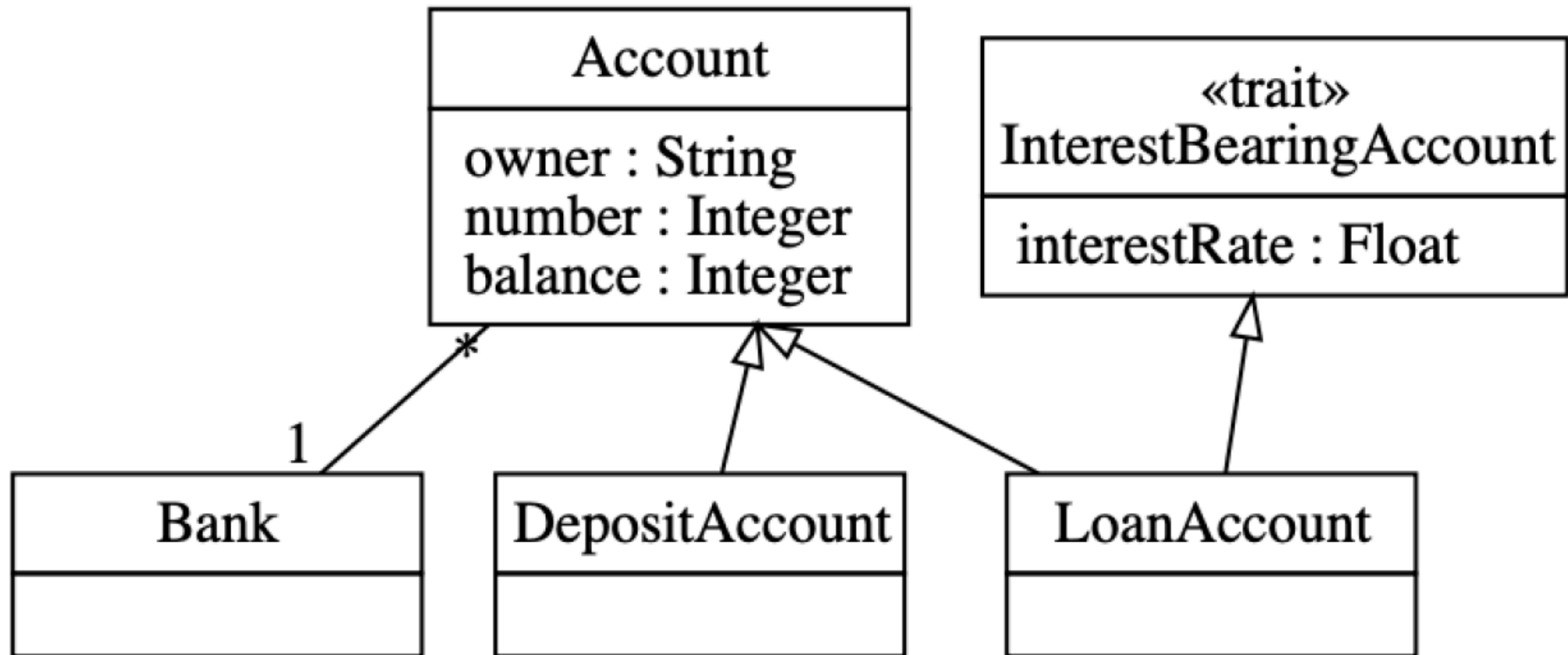
```
mixset Name2 class X {a;}  
class X {b;}
```

- The above works for attributes, associations, state machines, states, etc.

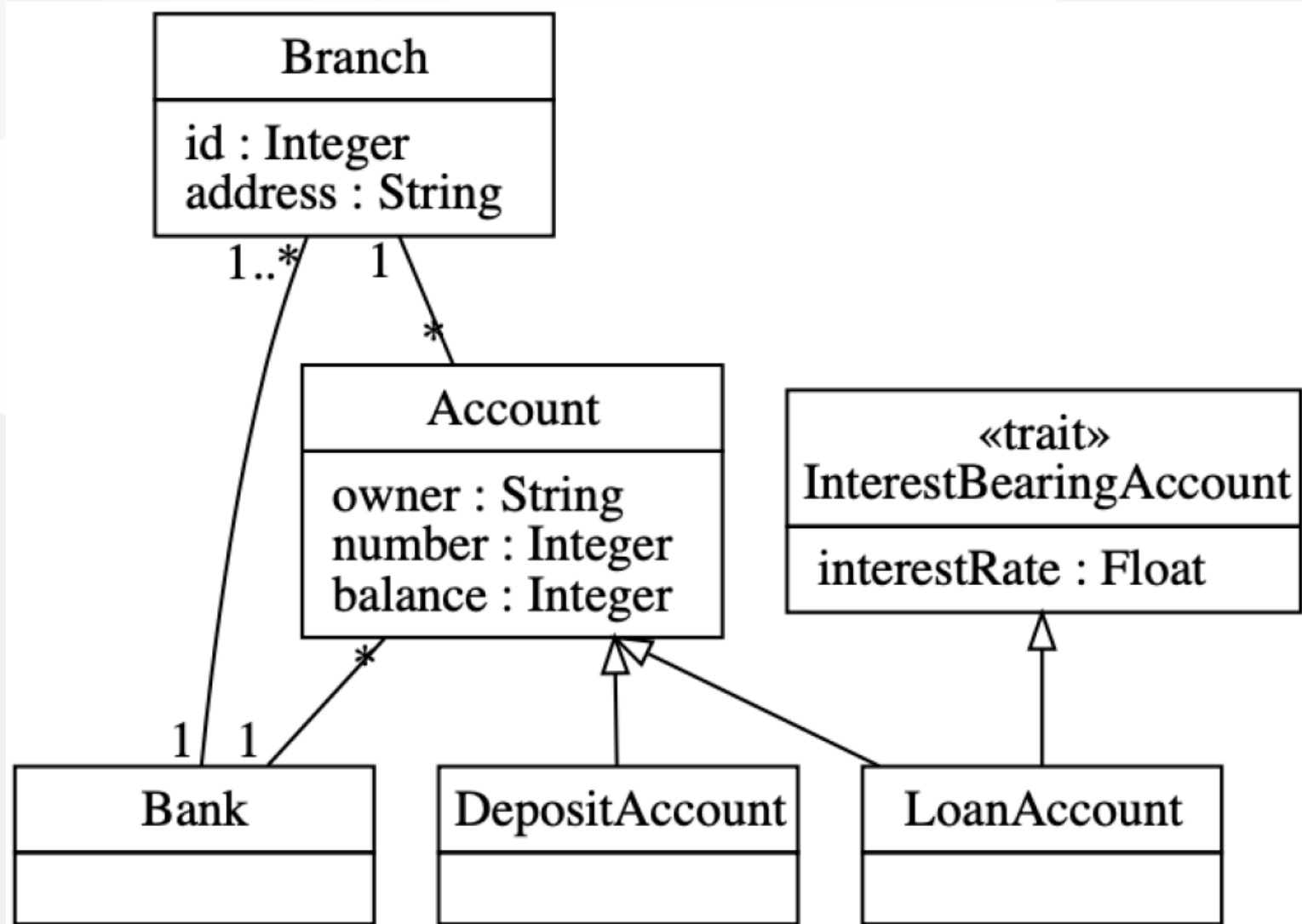
Motivating Example: Umlle Model/Code for Basic Bank

```
1  class Bank {  
2      1 -- * Account;  
3  }  
4  
5  class Account {  
6      owner; Integer number; Integer balance;  
7  }  
8  
9  trait InterestBearingAccount {  
10     Float interestRate;  
11 }  
12  
13 class DepositAccount {  
14     isA Account;  
15 }  
16  
17 class LoanAccount {  
18     isA Account, InterestBearingAccount;  
19 }
```


Class Diagram of Basic Bank Example:



Adding Optional Multi-branch Feature



Example: Multi-branch Umple Model/Code

```

1  class Bank {
2      1 -- * Account;
3      mixset Multibranch 1 -- 1..* Branch;
4  }
5
6  mixset Multibranch class Branch {
7      Integer id; String address;
8  }
9
10 class Account {
11     owner; Integer number; Integer balance;
12     mixset Multibranch * -- 1 Branch;
13 }
14
15 trait InterestBearingAccount {
16     Float interestRate;
17 }
18
19 class DepositAccount {
20     isA Account;
21     mixset OverdraftsAllowed {
22         Integer overdraftLimit;
23         isA InterestBearingAccount;
24     }
25 }
26
27 class LoanAccount {
28     isA Account, InterestBearingAccount;
29 }

```

Models T3 Tutorial: Umple - October 2020

Alternative Approach (same system)

```
1  class Bank {
2      1 -- * Account;
3  }
4
5  class Account {
6      owner; Integer number; Integer balance;
7  }
8
9  trait InterestBearingAccount {
10     Float interestRate;
11 }
12
13 class DepositAccount {
14     isA Account;
15     mixset OverdraftsAllowed {
16         Integer overdraftLimit;
17         isA InterestBearingAccount;
18     }
19 }
20
21 class LoanAccount {
22     isA Account, InterestBearingAccount;
23 }
24
25 mixset Multibranch {
26     class Bank {1 -- 1..* Branch}
27     class Branch {Integer id; String address;}
28     class Account {* -- 1 Branch}
29 }
```

Constraints on Mixsets

```
require [Mixset1 or Mixset2];
```

- Allowed operators
 - and, or, xor
 - not
 - n..m of {...}
- Parentheses allowed

```
opt X (means 0..1 of {X})
```

Case Study and Exercise 1: **Modifying the banking example**

- I will give you the text of the banking example and set up a task for you to:
 - Add the ability to have one or more account holders
 - Add the ability to have one or more co-signers

Case Study and Exercise 2: Dishwasher example

- We will start with the Dishwasher example in UmpleOnline
- We will use UmpleOnline's Task capability to ask you to split the Dishwasher example into two versions
 - A cheap version that only does normal wash and not fast wash
 - A full version that does everything
- Hint: Pull out the relevant state and transition for fast wash and wrap it in a mixset

Case Study 3: **Umple itself, written in Umple**

- We will look at:
 - Code in Github
 - Generated Architecture diagrams
 - Generated Javadoc
 - Sample master code
 - Sample test output
 - Sample code for generators (that replaced Jet)
 - UmpleParser (that replaced Antlr)

Unit Testing with UMPLE

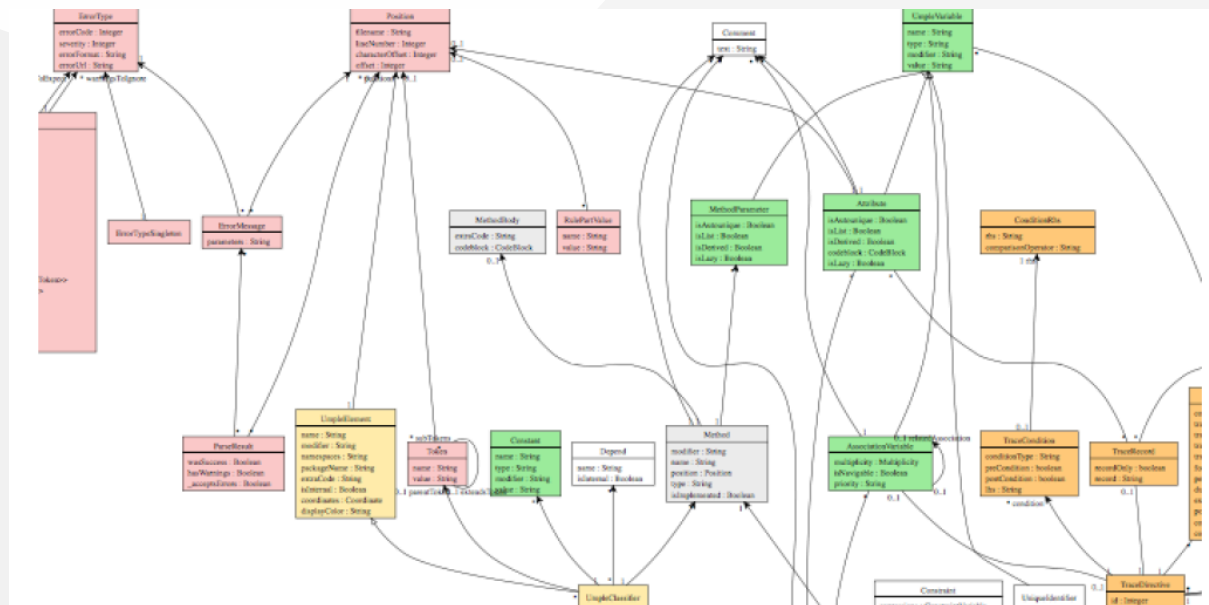
Unit Testing with Umple

- To see how to integrate Unit Testing with Umple, see the sample project at
 - <https://github.com/umple/umple/tree/master/sandbox>
- And the build script at
 - <https://github.com/umple/umple/blob/master/build/build.sandbox.xml>
- Command line from build directory

```
ant -f build.xml sandbox
```

A Look at How Umple is Written in Itself

- Source:
 - <https://github.com/umple/umple/tree/master/cruise.umple/src>
- Umple's own class diagram generated by itself from itself:
 - <http://metamodel.umple.org>
 - Colours represent key subsystems
 - Click on classes to see Javadoc, and then Umple Code



Testing: TDD with 100% pass always required

- Multiple levels: <https://cruise.eecs.uottawa.ca/qa/index.php>
- **Parsing tests:** basic constructs
- **Metamodel tests:** ensure it is populated properly
- E.g.
 - <https://github.com/umple/umple/blob/master/cruise.umple/test/cruise/umple/compiler/AssociationTest.java>
- **Implementation template tests:** to ensure constructs generate code that looks as expected
- **Testbed semantic tests:** Generate code and make sure it behaves the way it should

UMPLE issues list

UMPLE issues list

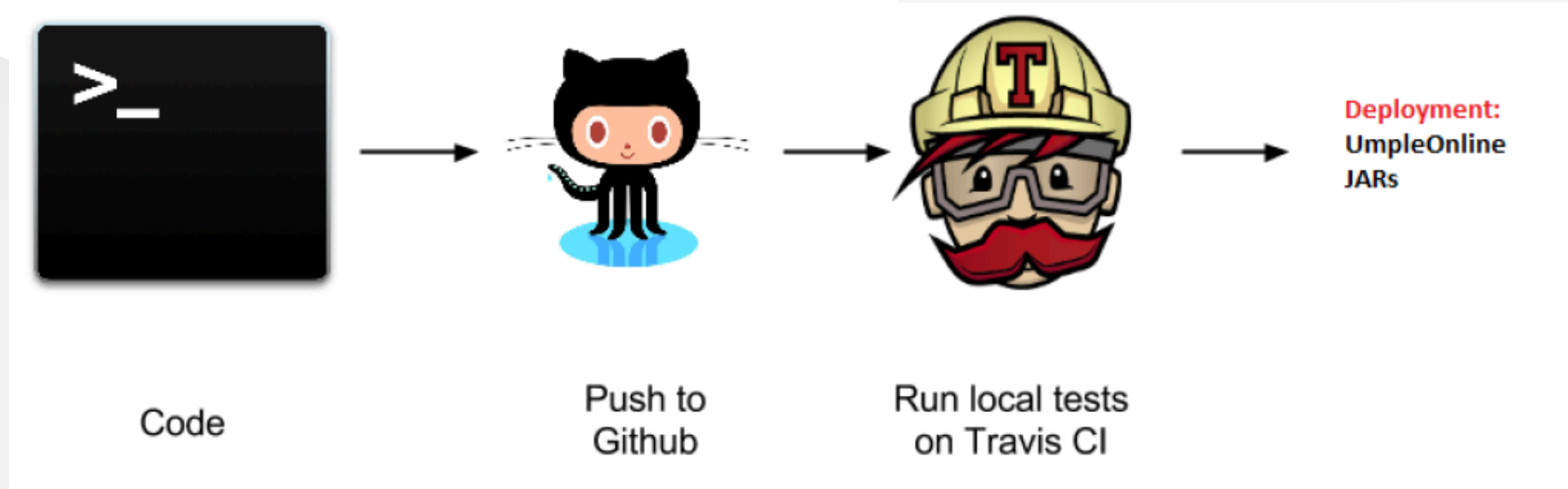
- Tagged by
- Priority
- Perceived difficulty
- Scale (bug, project, research project)
- Milestone (slow release)

<http://bugs.umple.org>

Using Umple with Builds and Continuous Integration

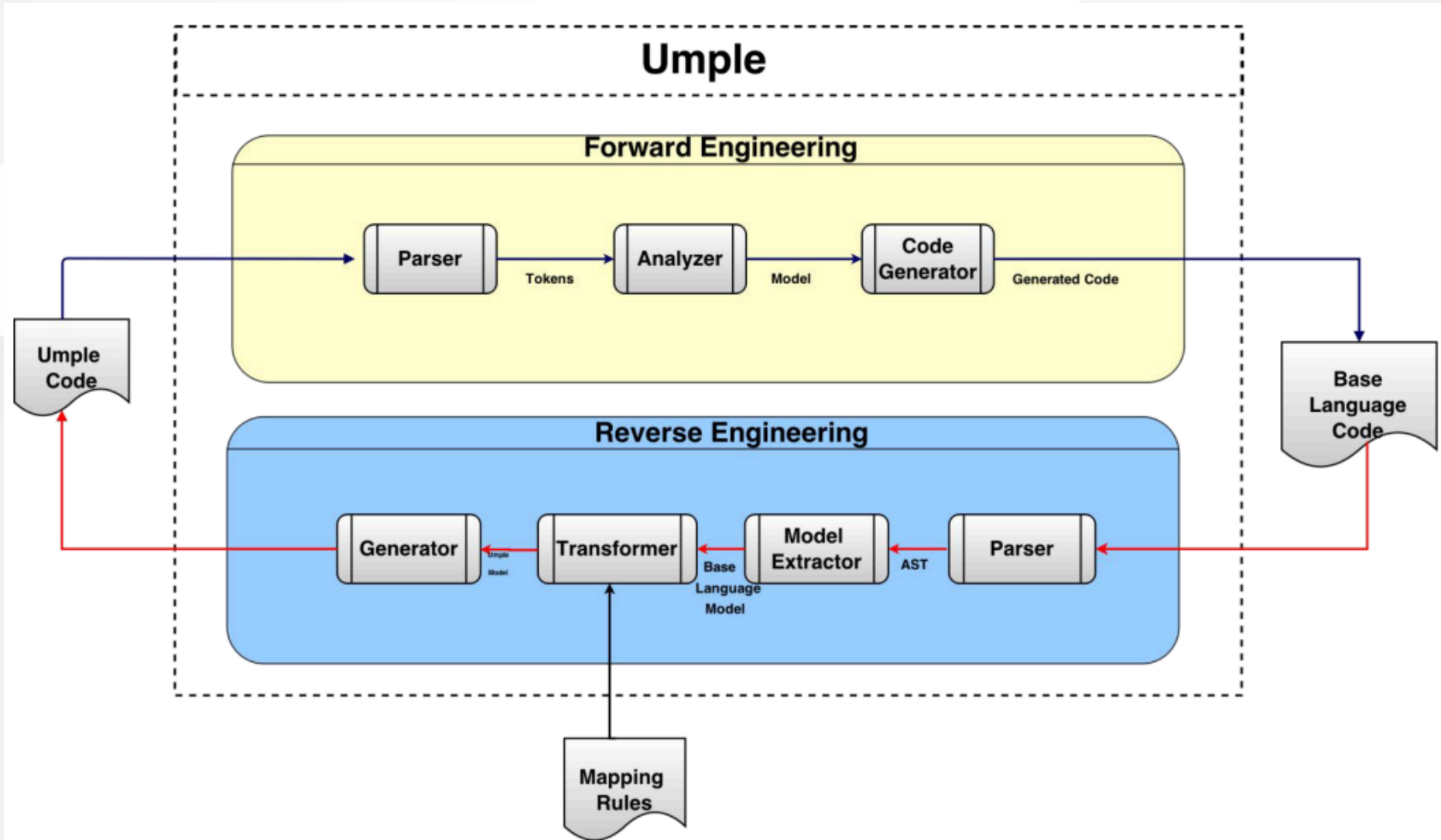
Using Umple with Builds and Continuous Integration

- Example build scripts
- Example [travis.yml](#)
- Umple's own [Travis](#) page



UMPLE's Architecture

Umple's Architecture



Umplification

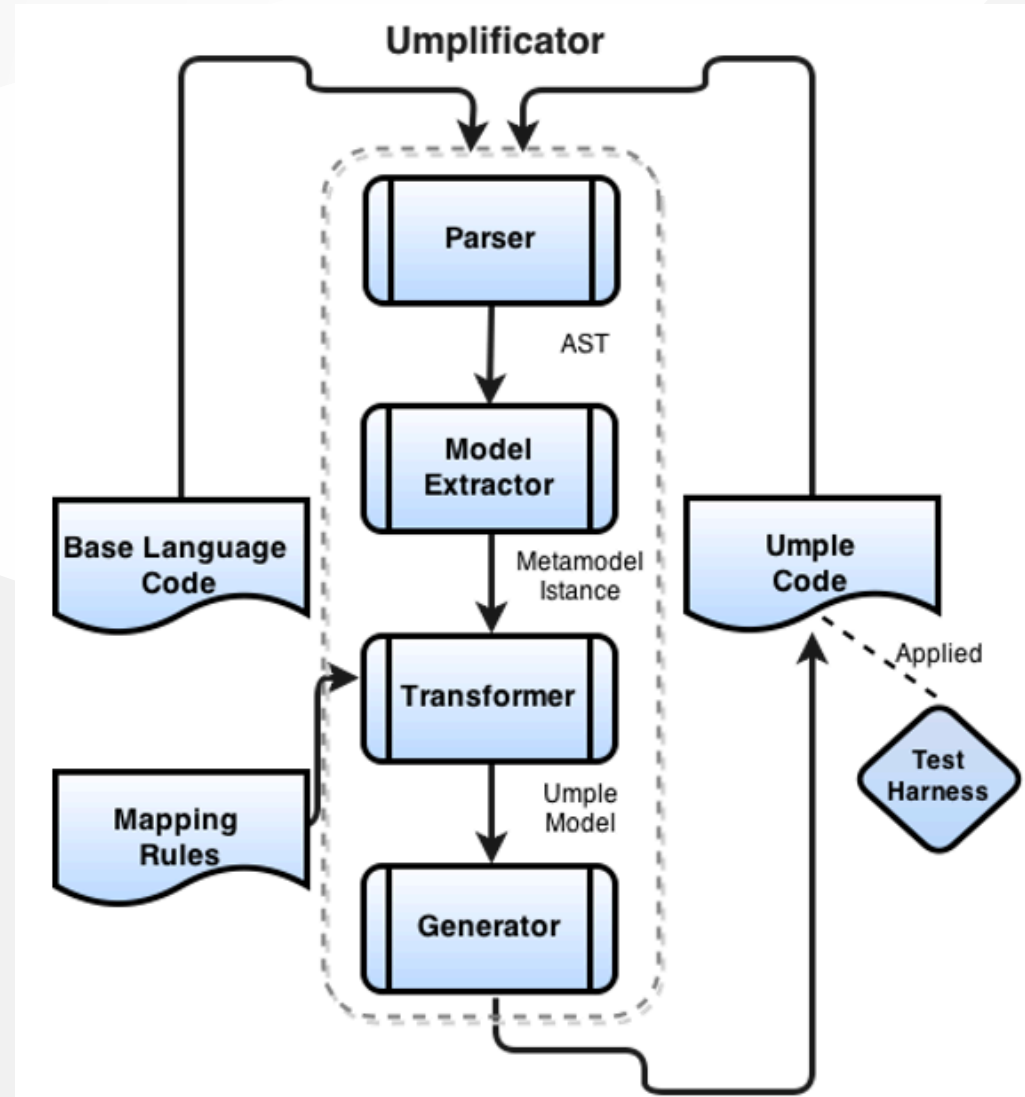
Umplification

- Umplification: 'amplication' + converting into Uml.
- Produces a program with behavior identical to the original one but written in Uml.
- Eliminates the distinction between code and model. Proceeds incrementally until the desired level of abstraction is achieved.

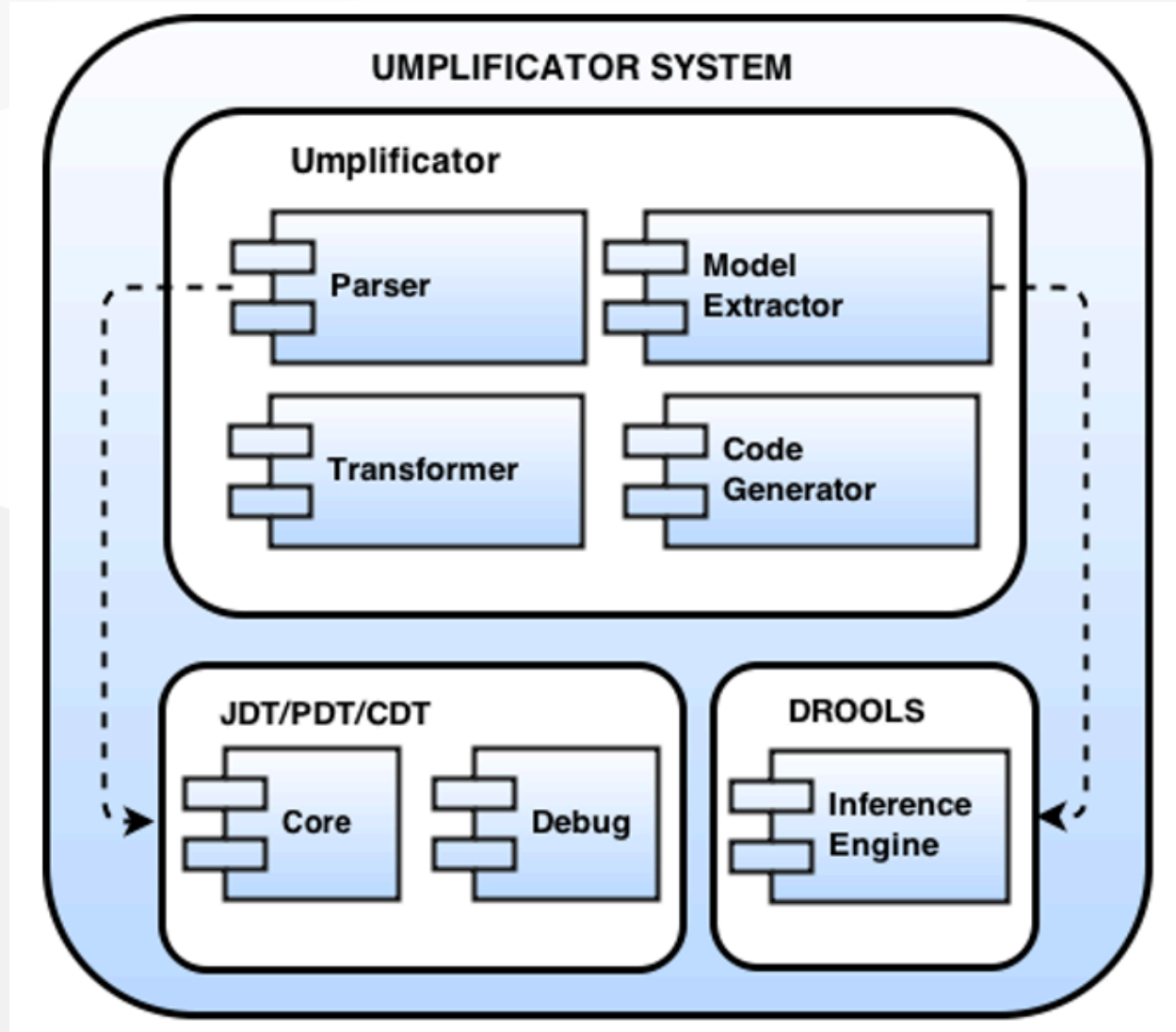
Uimplification: The Transformation Steps

- **Transformation 0:** Initial transformation
- **Transformation 1:** Transformation of generalization, dependency, and namespace declarations.
- **Transformation 2:** Analysis and conversion of many instance variables, along with the methods that use the variables.
 - **Transformation 2a:** Transformation of variables to UML/Uml attributes.
 - **Transformation 2b:** Transformation of variables in one or more classes to UML/Uml associations.
 - **Transformation 2c:** Transformation of variables to UML/Uml state machines.

Umplification Process



Umplificator Architecture



Umplification - Example

Person.java

```
1 package university;
2 public class Person {
3     public String getName() {return this.name;}
4     public void setName(String name){
5         this.name= name;
6     }
7 }
```


Umplification - Example

LISTING 3.2: Student.java

```

20 package university;
21
22 public class Student extends
    Person{
23
24     public static final int
        MAX_PER_GROUP = 10;
25     private int id;
26     private String name;
27     public Mentor mentor;
28
29     public Student(int id,String
        name){
30         id = id; name = name;
31     }
32     public String getName(){
33         String aName = name;
34         if (name == null) {
35             throw new RuntimeException("
                Error");
36         }
37         return aName;
38     }
39     public Integer getId() {
40         return id;
41     }
42     public void setId(Integer id) {
43         this.id = id;
44     }
45     public boolean getIsActive() {
46         return isActive;
47     }
48     public void setIsActive(boolean
        aIsActive) {
49         isActive = aIsActive;}
50     }
51     public Mentor getMentor() {
52         return mentor;
53     }
54     public void setMentor(Mentor
        mentor) {
55         this.mentor = mentor;
56     }
57 }

```

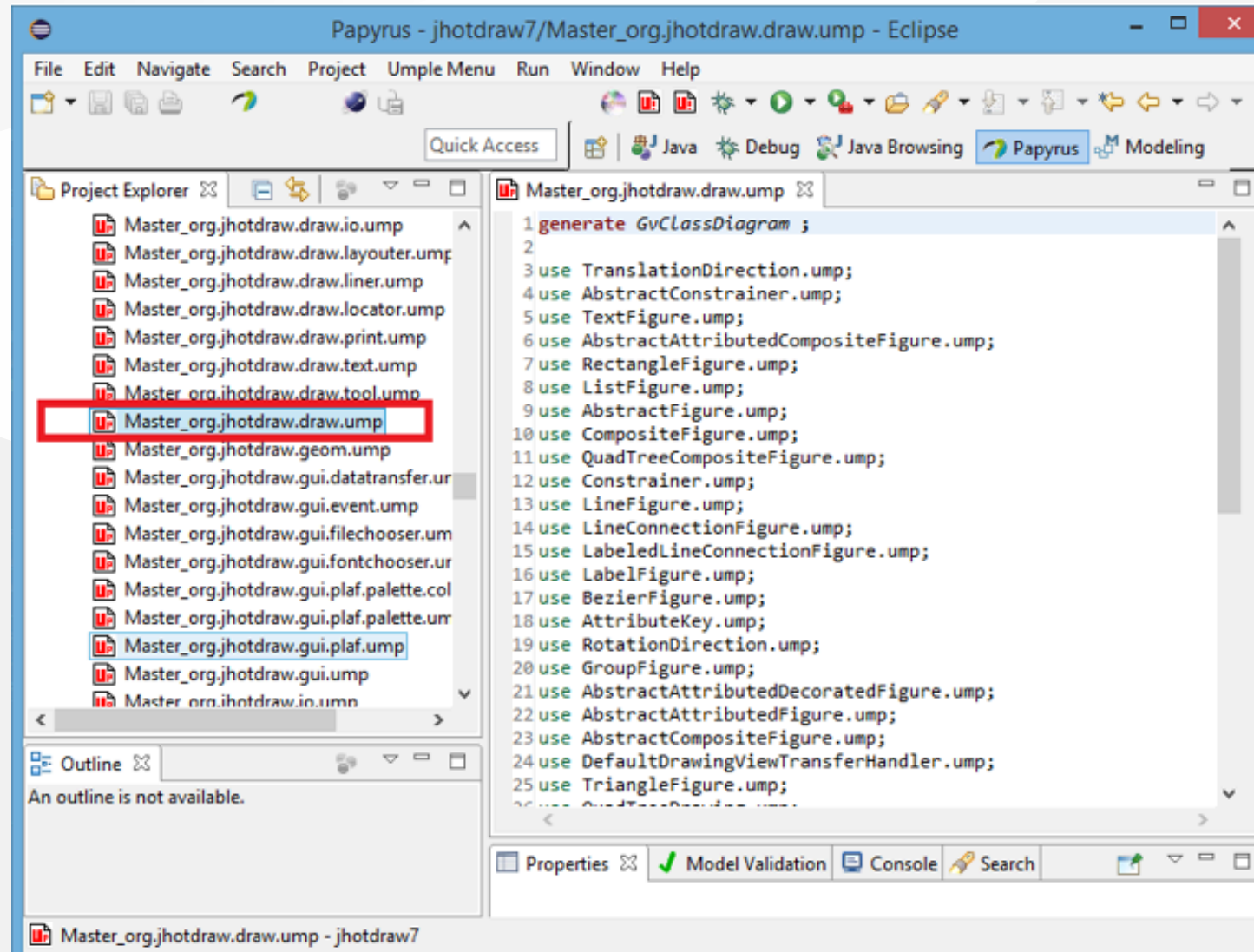
LISTING 3.3: Mentor.java

```

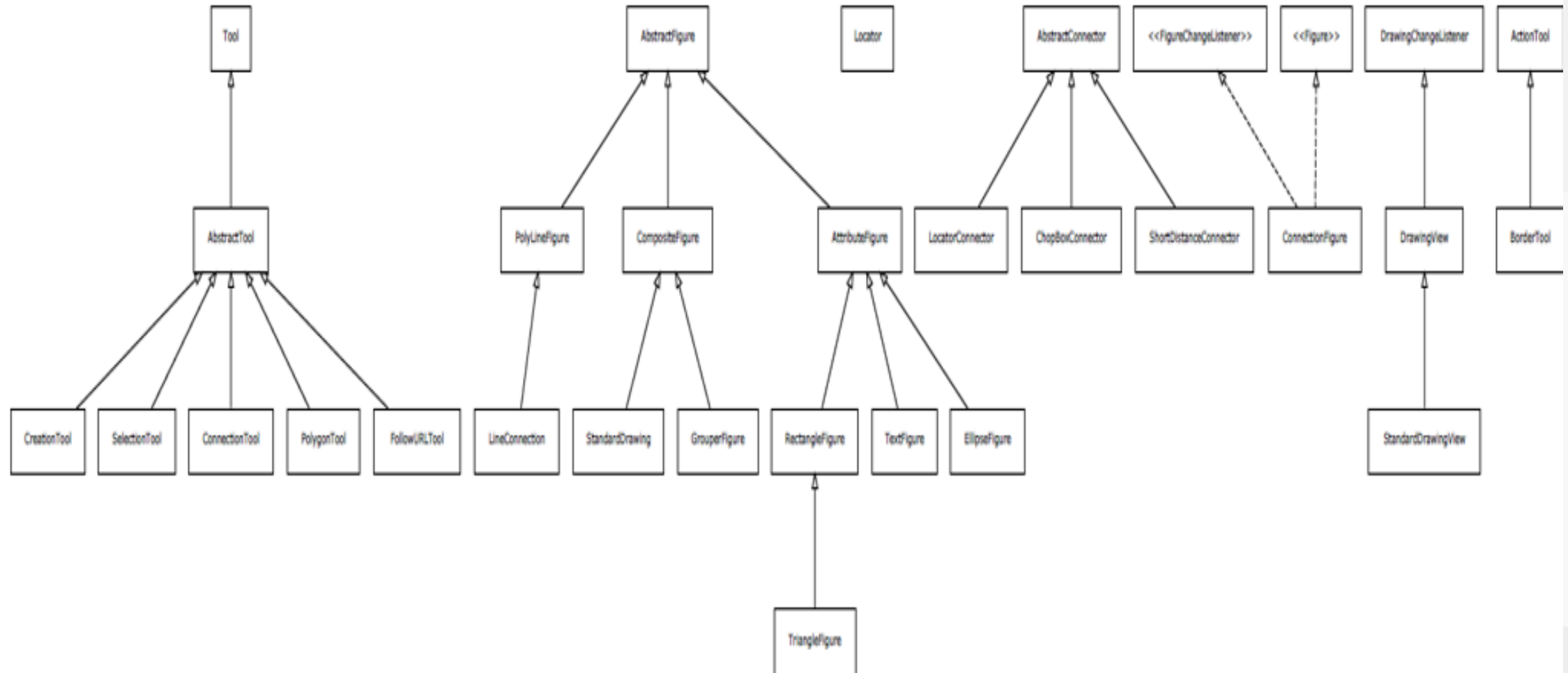
1 package university;
2 import java.util.Set;
3
4 public class Mentor extends
    Person{
5
6     Mentor() {}
7     public Set<Student> students;
8     public Set<Student> getStudents
        () {
9         return students;
10    }
11    public void setStudents (Set<
        Student>students) {
12        this.students = students;
13    }
14    public void addStudent( Student
        aStudent){
15        students.add(aStudent);
16    }
17    public void removeStudent(
        Student aStudent) {
18        students.remove(aStudent);
19    }
20    public String toString() {
21        return(
22            (name==null ? " " : name
23            ) + " " +
24            students.size()+ "
                students"
25        );
26    }

```

Systems umplified (JhotDraw 7.5.1)



Systems umplified (JhotDraw 7.5.1)



Systems unplified

- Weka
 - Associations unplified
- Args4J- Modernization
 - Original Args4j source code is composed of 61 classes and 2223 LOC.
 - Unplified Args4j source code is composed of 122 (2 per input class) umple files and 1980 LOC.
- # LOC in files containing modeling constructs (X.ump) is 312.
- # LOC in files with algorithmic/logic code (X code.ump) is 1668.

The developer must then translate 1518 lines of code rather than 2223 lines of code.

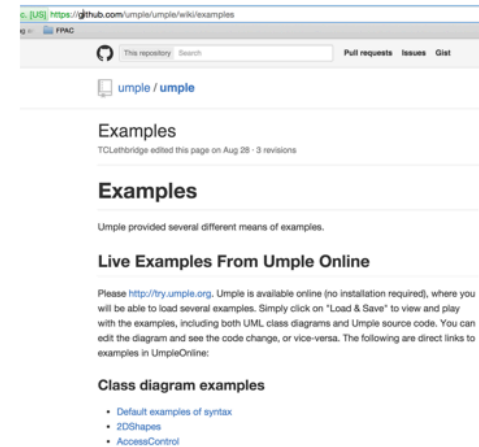
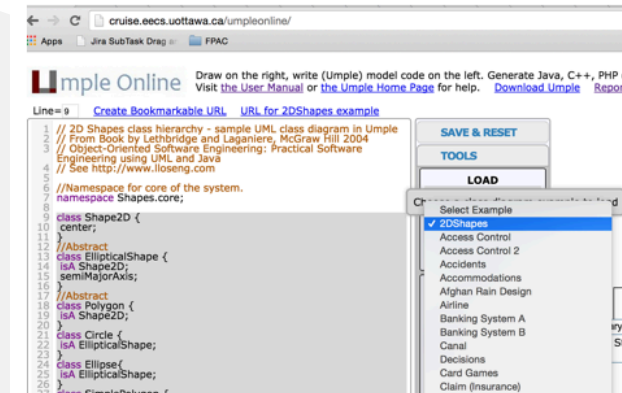
Conclusion

Conclusion

- Umlle
 - Is simple but powerful modeling tool
 - Generates state-of-the-art code
 - Enables agility + model-driven development
- We call the overall approach model-based programming

Umlpe Examples More ..

- <http://try.umlpe.org>
- <https://github.com/umlpe/umlpe/wiki/examples>
- <http://umpr.a4word.com/>
- <http://code.umlpe.org>
- <http://metamodel.umlpe.org>



Takeaway: Mixsets, Testing & Architecture

- Mixsets enable software product line engineering - conditionally include/exclude features
- UMPLE supports unit testing through generated test harnesses
- UMPLE integrates with build systems (Maven, Gradle) and CI pipelines
- Umplification: process of converting existing code into UMPLE models
- UMPLE's own architecture demonstrates model-based programming principles

References

References

- [UMPLE Tutorials](#)
- [UMPLE Github](#)
- [UMPLE Online](#)
- [UMPLE Documentation](#)
- [UMPLE CSI5112– February 2018](#)
- [Umple Tutorial: Models 2020 Web](#)
- [Umple Tutorial: Models 2020 Pdf](#)

References

- [Getting Started in UMPLE](#)
- [Experiential Learning for Software Engineering Using Agile Modeling in Umple \(Youtube\)](#)
- [Experiential Learning for Software Engineering Using Agile Modeling in Umple \(Slide\)](#)
- [Tomassetti Code Generation](#)

